

Mall Lift Machine Controller Chip Project

A Complete Beginner's End-to-End ASIC Design Workflow Guide for an Elevator Controller

Target Architecture: Generic ASIC (Gate-Level Netlist Standard Cells)

Language: Verilog HDL (IEEE 1364-2001)

Platform: EDA Playground Simulation & Verification environment

Document Version: 1.0.0 (Production-Ready)

1. Introduction & The Lift Controller ASIC Concept

Scaling up from a traffic light controller, a lift (elevator) controller introduces interactive external inputs (button presses) that must be captured and acted upon by a Finite State Machine (FSM). In a busy shopping mall environment, elevator chips manage real-time request tracking, direction changes, motor driver switches, and door security controls. This document takes you from system specifications to a functional, synthesizable Verilog implementation tailored for EDA Playground simulation.

ASIC Front-End Implementation Strategy

Our design targets a 3-floor mall elevator system (Floor 0: Ground/Parking, Floor 1: Retail shops, Floor 2: Food Court/Cinema). The design maintains state variables for the current floor location, active movement directions, and door open/closed status vectors.

2. System Specification & Architectural Design

The controller must dynamically process inputs from buttons located both inside the elevator car and on the individual mall floors. To keep this beginner-friendly, we model the system requests as a destination target floor input.

FSM State Definitions

- **IDLE (3'b000)**: Lift stationary at a floor with doors closed. Waiting for a destination request.
- **MOV_UP (3'b001)**: Elevator motor active, moving upwards to a higher floor.
- **MOV_DN (3'b010)**: Elevator motor active, moving downwards to a lower floor.
- **DOOR_OP (3'b011)**: Stationary at the target floor; doors actively opening.
- **DOOR_CL (3'b100)**: Stay duration reached; doors actively closing.

Output Representation Matrix

FSM State	Motor Status (2 bits)	Door Status (2 bits)	Floor Indicator Displays (2 bits)
IDLE	2'b00 (Stopped)	2'b00 (Closed)	Current Floor Vector
MOV_UP	2'b01 (Moving Up)	2'b00 (Closed)	Current Floor Vector
MOV_DN	2'b10 (Moving Down)	2'b00 (Closed)	Current Floor Vector
DOOR_OP	2'b00 (Stopped)	2'b01 (Open)	Target Floor Vector
DOOR_CL	2'b00 (Stopped)	2'b10 (Closing)	Target Floor Vector

3. Verilog RTL Implementation (`lift\_controller.v`)

This Verilog module implements a safe design using parameterized coding practices. Save this into the design pane of EDA Playground.

```
// =====
// DESIGN UNIT: lift_controller.v
// TYPE: RTL Hardware Description
// ENCODING: IEEE 1364-2001 Verilog Standard
// =====

module lift_controller (
    input wire clk,           // System clock
    input wire rst_n,        // Asynchronous active-low reset
    input wire [1:0] req_floor, // Requested floor from user (0, 1, or 2)
    input wire req_valid,     // Asserted high when a new button is pressed

    output reg [1:0] current_floor, // Current location binary output
    output reg [1:0] motor_state,   // 00=Stop, 01=Up, 10=Down
    output reg [1:0] door_state     // 00=Closed, 01=Open, 10=Closing
);

    // FSM State Encodings
    parameter STATE_IDLE    = 3'b000;
    parameter STATE_MOV_UP  = 3'b001;
    parameter STATE_MOV_DN  = 3'b010;
    parameter STATE_DOOR_OP = 3'b011;
    parameter STATE_DOOR_CL = 3'b100;

    // Internal State Registers
    reg [2:0] current_state;
    reg [2:0] next_state;

    // Internal register to lock and store requested destination floor
    reg [1:0] target_floor;

    // Step counter for transitions (simulates travel and door times)
    reg [3:0] delay_counter;

    // -----
    // BLOCK 1: Sequential Process (State & Internal Registers)
    // -----
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            current_state <= STATE_IDLE;
            current_floor <= 2'b00; // Reset to Ground Floor
            target_floor <= 2'b00;
            delay_counter <= 4'd0;
        end else begin
            current_state <= next_state;

            // Latch new target floor if valid request arrives while idle
            if (current_state == STATE_IDLE && req_valid) begin
                target_floor <= req_floor;
            end

            // Simulation Counter logic
            if (current_state != next_state) begin
                delay_counter <= 4'd0;
                // Update floor indicator when reaching destinations
                if (current_state == STATE_MOV_UP) current_floor <= current_floor + 1'b1;
            end
        end
    end
endmodule
```

```

        if (current_state == STATE_MOV_DN) current_floor <= current_floor - 1'b1;
    end else begin
        delay_counter <= delay_counter + 1'b1;
    end
end
end

// -----
// BLOCK 2: Combinational Process (Next State Logic)
// -----
always @(*) begin
    case (current_state)
        STATE_IDLE: begin
            if (req_valid && (req_floor > current_floor))
                next_state = STATE_MOV_UP;
            else if (req_valid && (req_floor < current_floor))
                next_state = STATE_MOV_DN;
            else if (req_valid && (req_floor == current_floor))
                next_state = STATE_DOOR_OP;
            else
                next_state = STATE_IDLE;
        end

        STATE_MOV_UP: begin
            // Simulate time taken to move up one floor (3 cycles)
            if (delay_counter == 4'd3) begin
                if (current_floor + 1'b1 == target_floor)
                    next_state = STATE_DOOR_OP;
                else
                    next_state = STATE_MOV_UP; // Keep moving if multi-floor travel
            end else begin
                next_state = STATE_MOV_UP;
            end
        end

        STATE_MOV_DN: begin
            // Simulate time taken to move down one floor (3 cycles)
            if (delay_counter == 4'd3) begin
                if (current_floor - 1'b1 == target_floor)
                    next_state = STATE_DOOR_OP;
                else
                    next_state = STATE_MOV_DN;
            end else begin
                next_state = STATE_MOV_DN;
            end
        end

        STATE_DOOR_OP: begin
            // Hold doors open for 3 cycles for mall passengers
            if (delay_counter == 4'd3)
                next_state = STATE_DOOR_CL;
            else
                next_state = STATE_DOOR_OP;
        end

        STATE_DOOR_CL: begin
            // Doors take 2 cycles to securely latch close

```

```

        if (delay_counter == 4'd2)
            next_state = STATE_IDLE;
        else
            next_state = STATE_D00R_CL;
        end

        default: next_state = STATE_IDLE;
    endcase
end

// -----
// BLOCK 3: Combinational Process (Output Drivers)
// -----
always @(*) begin
    // Safe Defaults
    motor_state = 2'b00; // Stopped
    door_state  = 2'b00; // Closed

    case (current_state)
        STATE_IDLE: begin
            motor_state = 2'b00;
            door_state  = 2'b00;
        end
        STATE_MOV_UP: begin
            motor_state = 2'b01; // Moving Up
            door_state  = 2'b00;
        end
        STATE_MOV_DN: begin
            motor_state = 2'b10; // Moving Down
            door_state  = 2'b00;
        end
        STATE_D00R_OP: begin
            motor_state = 2'b00;
            door_state  = 2'b01; // Open
        end
        STATE_D00R_CL: begin
            motor_state = 2'b00;
            door_state  = 2'b10; // Closing
        end
    endcase
end

endmodule

```

4. Functional Verification Testbench (`tb\_lift\_controller.v`)

This automated testbench acts as a virtual shopping mall passenger pressing buttons on different floors to see how the elevator behaves.

```

// =====
// VERIFICATION UNIT: tb_lift_controller.v
// TYPE: Behavioral Testbench Environment
// =====

`timescale 1ns/1ps

module tb_lift_controller;

    reg clk;
    reg rst_n;
    reg [1:0] req_floor;
    reg req_valid;

    wire [1:0] current_floor;
    wire [1:0] motor_state;
    wire [1:0] door_state;

    // Instantiate UUT
    lift_controller uut (
        .clk(clk),
        .rst_n(rst_n),
        .req_floor(req_floor),
        .req_valid(req_valid),
        .current_floor(current_floor),
        .motor_state(motor_state),
        .door_state(door_state)
    );

    // 10ns Clock Generation
    always begin
        #5 clk = ~clk;
    end

    // Test Scenarios Execution
    initial begin
        // System Init
        clk = 0;
        rst_n = 0;
        req_floor = 2'b00;
        req_valid = 0;

        // Release system reset
        #20 rst_n = 1;

        // Scenario A: Passenger on Floor 2 presses button
        #15;
        req_floor = 2'd2;
        req_valid = 1;
        #10; // Hold for 1 clock pulse
        req_valid = 0;

        // Allow FSM time to move up, open doors, and go back to IDLE
        #150;

        // Scenario B: Passenger inside on Floor 2 calls to go down to Floor 1

```

```

        req_floor = 2'd1;
        req_valid = 1;
        #10;
        req_valid = 0;

        #100;
        $display("[LIFT_VERIFICATION] All Mall scenarios simulated successfully.");
        $finish;
    end

    // Direct Monitor Print
    initial begin
        $monitor("Time=%0dns | Reset=%b | TargetReq=%d | Car Floor=%d | Motor Status=%b |
Doors=%b",
                $time, rst_n, req_floor, current_floor, motor_state, door_state);
    end

    // Waveform Configuration for EPWave
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_lift_controller);
    end

endmodule

```

5. Running the Lift Controller Project on EDA Playground

- 1 Paste the **Verification Testbench** code into the left pane window and the **RTL Design** into the right pane window.
- 2 Set the Language profile toolbox configuration drop-down menus to **Verilog/SystemVerilog** and choose **Icarus Verilog** as your target compilation engine tool.
- 3 Check the box to automatically initialize **EPWave** layout graphs post-compilation, and click **Run**.
- 4 Watch the text monitor logs. You will explicitly trace the system changing states: checking target parameters, starting upward traction drivers (`motor_state = 01``), stepping up floor index boundaries, safely activating structural hatch door locks (`door_state = 01``), and dropping back down to unforced rest profiles (`IDLE``).